

Programación declarativa

- Haskell •

Doble grado en Ingeniería Informática y Matemáticas

Universidad Complutense de Madrid

Óscar Martín — Curso 2026/27

1. Introducción a Haskell

- G. Hutton: “*Haskell is a purely functional language that allows programmers to rapidly develop clear, concise and correct software.*”

URLs *oficiales*

- <https://www.haskell.org/>
- <https://www.haskell.org/downloads/>
- <https://play.haskell.org/> (para programar en Haskell *online*)
- <https://hoogle.haskell.org/> (buscador)

Herramientas

- Vamos a usar GHC, Glasgow Haskell Compiler.
- GHCup es el instalador. Instala:
 - ghc (compilador)
 - ghci (interactivo)
 - Haskell Language Server (HLS, es opcional, pero instálalo)
- VSCode + Haskell (extensión) + Haskell Syntax Highlighting (extensión)
 - O cualquier otro editor.

Ejemplo: suma de una lista

```
suma :: [Int] -> Int
suma [] = 0
suma (n:ns) = n + suma ns
```

- En Haskell escribimos expresiones, no instrucciones.
- Las expresiones se evalúan: $\text{suma } [1, 2, 3] \rightsquigarrow 1 + \text{suma } [2, 3]$
 $\rightsquigarrow 1 + (2 + \text{suma } [3]) \rightsquigarrow 1 + (2 + (3 + \text{suma } [])) \rightsquigarrow 1 + (2 + (3 + 0))$
 $\rightsquigarrow 1 + (2 + 3) \rightsquigarrow 1 + 5 \rightsquigarrow 6.$
- Razonamiento ecuacional: $\text{suma } [x] = \text{suma } (x:[]) = x + \text{suma } [] = x + 0 = x.$

Algunas órdenes de ghci

- `:load nombre_de_archivo`
– o bien, directamente, `ghci nombre_de_archivo`
- `:reload`
- `:type expresión`
- `:info nombre de función, tipo o clase`
- `:quit` (también `ctrl+D`)
- `:?`

Se pueden abreviar a su primera letra: `:l`, `:r`, `:t`, `:i`, `:q`.

Ejemplo: *quicksort*

```
qsort :: [Int] -> [Int]
qsort [] = []
qsort (x:xs) = qsort menores ++ [x] ++ qsort mayores
               where menores = filter (<=x) xs
                     mayores = filter (>x) xs
```

- El tipo se puede generalizar: `qsort :: Ord a => [a] -> [a]`.
- Compárese con implementaciones imperativas.

Ejercicios

1. Modifica `qsort` para que ordene de mayor a menor.
2. Basándote en la definición de `suma`, escribe una función `todos :: [Bool] -> Bool`, que dice si todos los elementos de una lista de valores booleanos son ciertos. Puedes usar el operador de conjunción de Haskell: `&&`.
3. Escribe tus soluciones a los ejercicios en un archivo `.hs` y pruébalas en `ghci`.
4. Añade esto a tu archivo `.hs`, compílalo con `ghc nombre_de_archivo` y ejecútalo:

```
main :: IO ()
main = do
  print (todos [True, True, False, True])
  print (qsort [3, 1, 4, 1, 5, 9, 2, 6])
```